

CLENZY

Property Management System

Assistant IA Architecture

& Documentation Technique

Core agent · 27 tools · KB RAG pgvector · Memory · Briefings · Vision · Workflows

Date : **27 mai 2026**

Version : **1.0**

Cible : **Équipe technique Clenzy**

Table des matières

1. Vue d'ensemble	4
2. Architecture globale	5
3. Modèle de données	7
4. Core Agent : Orchestrator + LLM Provider	9
5. Tool framework	11
6. Inventaire des 27 tools	12
7. Knowledge Base RAG (pgvector)	14
8. Memory long-terme	16
9. Simulations pricing & calendrier	17
10. Workflows procéduraux	18
11. Briefings proactifs	19
12. Vision (images)	21
13. Frontend chat	22
14. Sécurité & multi-tenant	23
15. Configuration & déploiement	24
16. Limites connues & TODOs	25

1. Vue d'ensemble

L'**Assistant IA Clenzy** est un agent conversationnel multimodal intégré au PMS (Property Management System), conçu pour automatiser les tâches répétitives, fournir des analyses métier en temps réel et exécuter des actions concrètes via le langage naturel.

Capacités métier

- **Comprendre** — Questions métier en FR/EN/AR sur propriétés, réservations, finances, opérationnel
- **Exécuter** — Actions concrètes : créer intervention, assigner technicien, bloquer calendrier
- **Simuler** — Scénarios « what-if » (changement de prix, blocage calendrier) avec modèle économique
- **Mémoriser** — Préférences et faits long-terme (4 scopes : preference, fact, goal, project)
- **Citer** — Documentation Clenzy via RAG pgvector — anti-hallucination strict
- **Orchestrer** — Workflows multi-étapes guidés (onboarding, clôture, préparation saison)
- **Envoyer** — Briefings proactifs daily/weekly via 3 canaux (in-app, email, WhatsApp)
- **Analyser** — Images (factures, dégradations, plans) via Claude Vision
- **Suggérer** — Navigation contextuelle vers les bonnes pages du PMS

Chiffres clés

27	8	65	~46 KB
Tools (agents)	Migrations DB	Fichiers Java	Orchestrator

Stack technique

Couche	Technologie
LLM	Anthropic Claude Sonnet 4 (vision, agent) + Haiku 4.5 (briefings)
Embeddings	Voyage AI voyage-3-lite (1024d) ou OpenAI text-embedding-3-small
Re-ranking	Voyage rerank-2 (cross-encoder) ou NoOp (fallback)
Vector store	PostgreSQL 16 + pgvector (vector(1024), ivfflat cosine)
Streaming	Server-Sent Events (pool dédié 10-100 threads)

Couche	Technologie
Persistence	JPA/Hibernate + Liquibase (migrations 0143-0150)
Cache	Redis (météo 3h, holidays, RAG hot queries)
Schedulers	Spring @Scheduled (6 schedulers : briefing, retry, memory, vision...)
Frontend	React 18 + TypeScript + MUI + EventSource SSE consumer
Storage images	S3 ou BYTEA (PhotoStorageService selon profile)

✓ L'assistant est **fail-soft systématique** : si le RAG est down, les briefings échouent, ou la clé API manque, l'utilisateur reçoit toujours une réponse intelligible avec dégradation gracieuse — jamais un écran blanc ou une exception non gérée.

2. Architecture globale

L'architecture suit un pattern **3-tier** : un frontend React qui consomme un flux SSE, un backend Spring Boot orchestré par **AgentOrchestrator** (cœur du système), et des services externes (Anthropic, Voyage, OpenMeteo, PostgreSQL pgvector, Redis).

Diagramme de composants

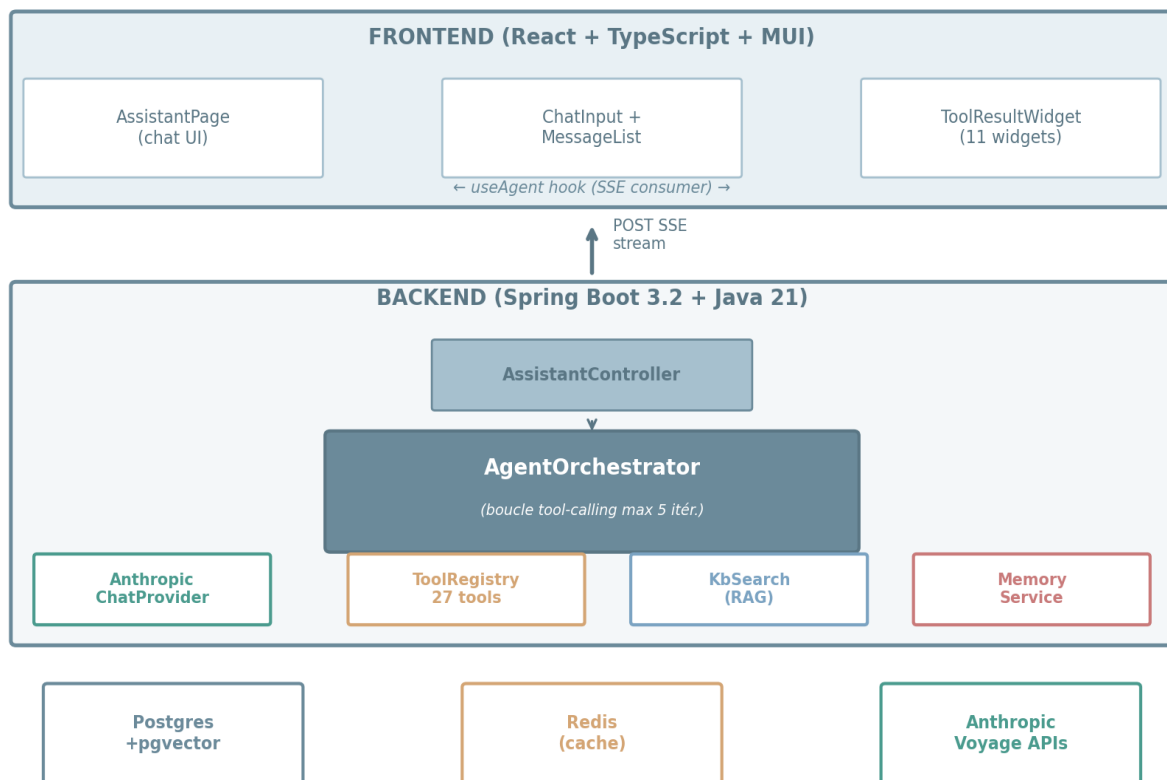


Figure 1 : Vue d'ensemble des composants frontend + backend + services externes

Flow end-to-end d'un message

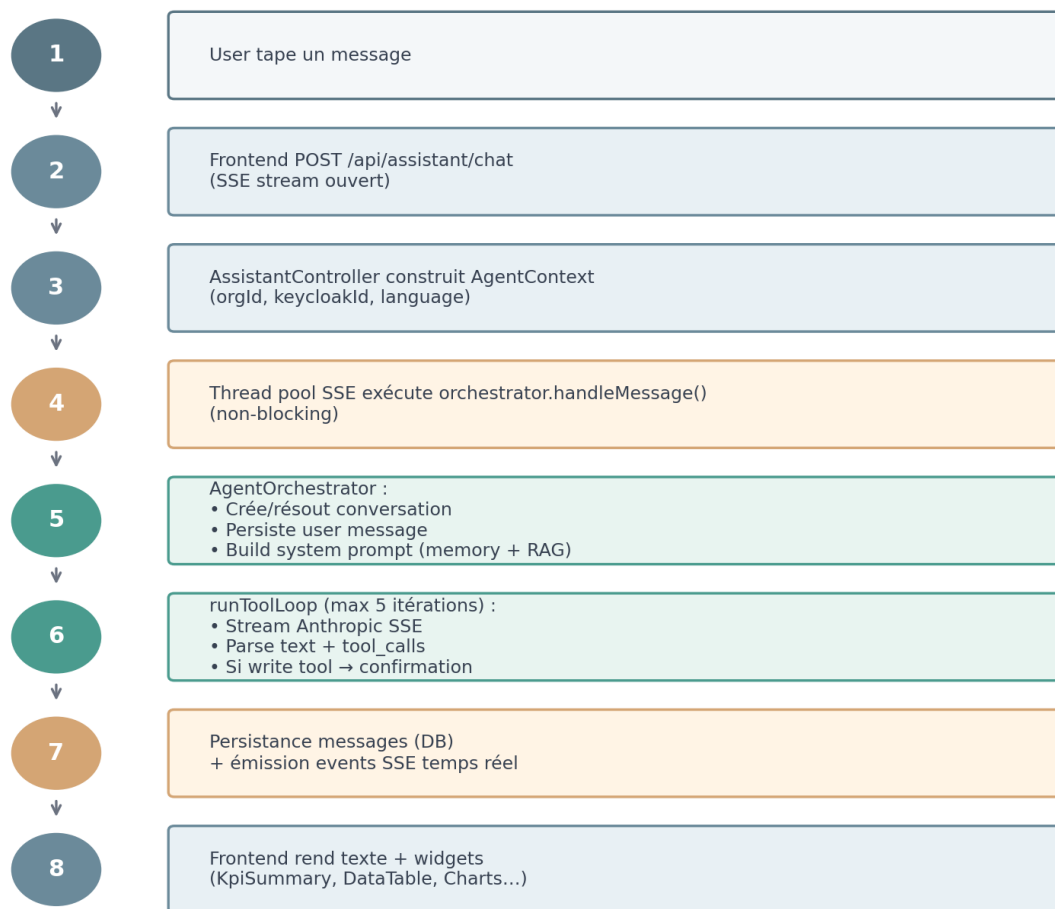


Figure 2 : Séquence complète de bout en bout (Frontend → Backend → LLM → Tools → Réponse)

3. Modèle de données

12 tables principales structurent la persistance de l'assistant. Toutes (sauf les tables admin cross-tenant) déclarent un **filtre Hibernate `organizationFilter`** activé automatiquement par le `TenantFilter` de la Spring Security chain. **Aucune fuite cross-tenant n'est possible** via les repositories standards.

Table	Colonnes clés	Index/Contraintes
assistant_conversation	id, organization_id, keycloak_id, title, model, archived_at	PK + (keycloak_id, created_at desc)
assistant_message	id, conversation_id, role, content, tool_calls JSONB, attachments JSONB, tokens	FK + (conversation_id, created_at asc)
assistant_memory	id, keycloak_id, memory_key, memory_value, scope, embedding vector(1024)	UNIQUE (kc_id, key) + ivfflat cosine
assistant_briefing_pref	keycloak_id, frequency, time_local, timezone, channels JSONB	UNIQUE (keycloak_id)
assistant_briefing_log	keycloak_id, briefing_date, status, conversation_id, channels	UNIQUE (kc_id, date) → idempotence
assistant_workflow_run	keycloak_id, workflow_id, current_step_idx, collected_data JSONB, version	Optimistic locking @Version
kb_document	organization_id (nullable=global), title, source_path, content_hash	UNIQUE (source_path, org_id)
kb_chunk	document_id, chunk_index, content, embedding vector(1024)	ivfflat cosine, FK doc_id
org_ai_api_key	organization_id, provider, api_key (AES-256), model_override	UNIQUE (org_id, provider)
org_whatsapp_template	organization_id, template_key, body, variables JSONB	UNIQUE (org_id, key)
org_vision_alert	organization_id (UNIQUE), monthly_token_threshold, last_alerted_at	Opt-in alerts par org
property_elasticity_estimate	property_id, elasticity, sample_size, computed_at	Cache empirique recalculé

Migrations Liquibase

ID	Description
0143	CREATE EXTENSION vector (pgvector)

ID	Description
0144	tables kb_document + kb_chunk
0145	colonne embedding vector(1024) sur assistant_memory
0146	colonnes last_accessed_at + expires_at sur assistant_memory
0147	tables property_pricing_config + property_elasticity_estimate
0148	table org_whatsapp_template
0149	table org_vision_alert
0150	colonne version (optimistic locking) sur assistant_workflow_run

■ **Point de vigilance** : les *native queries* bypassent le filtre Hibernate. Toute query native sur une table tenant-scoped **DOIT** inclure manuellement WHERE organization_id = :orgId en paramètre. Vérifié pour AssistantMemoryRepository.searchByCosineSimilarity() suite à un audit critique.

4. Core Agent : Orchestrator + LLM Provider

AgentOrchestrator (cœur du système)

Fichier : server/src/main/java/com/clenzy/service/agent/AgentOrchestrator.java (~46 KB)

Constantes clés :

Constante	Rôle
MAX_TOOL_ITERATIONS = 5	Safety net contre boucles infinies LLM ↔ tools
MAX_TOKENS_PER_TURN = 4096	Limite réponse LLM par tour
DEFAULT_TEMPERATURE = 0.3	Mode déterministe (moins de créativité)
MAX_MEMORY_ENTRIES = 30	Memory entries injectées dans system prompt
RAG_TOP_K = 4	Chunks KB auto-injectés à chaque message
RAG_RELEVANCE_MIN = 0.70	Seuil minimum cosine similarity pour injection

Boucle interne runToolLoop()

```
for iter in 0..MAX_TOOL_ITERATIONS:
    outcome = streamOneTurn(request, conversation, apiKey, consumer)
    if outcome.isError(): emit error → return
    persist(assistantMessage(text + tool_calls + tokens))
    if outcome.toolCalls.isEmpty():
        emit done → return
    if anyToolRequiresConfirmation(outcome.toolCalls):
        pendingToolStore.add(toolCallId, futureHistory)
        emit tool_confirmation_request → PAUSE
    for toolCall in outcome.toolCalls:
        handler = registry.find(toolCall.name())
        result = handler.execute(toolCall.args(), context)
        persist(toolMessage(toolCallId, result))
        request.appendMessage(toolMessage)
```

AnthropicChatProvider (LLM driver)

Wrapper SSE pour l'API Anthropic Messages, avec parsing en temps réel des events stream (message_start, content_block_delta, tool_use...). Supporte le BYOK (clé API par organisation, encryptée AES-256 dans org_ai_api_key) avec fallback sur la clé plateforme ANTHROPIC_API_KEY.

Event Anthropic SSE	Action interne
message_start	capture usage.input_tokens initial

Event Anthropic SSE	Action interne
content_block_start (text)	initialise accumulateur texte
content_block_start (tool_use)	initialise accumulateur args tool
content_block_delta (text_delta)	accumule + émet ChatEvent.TextDelta
content_block_delta (input_json_delta)	accumule args tool (string concat)
content_block_stop	flush du tool_use → ChatEvent.ToolCallRequest
message_delta	capture stop_reason final + output_tokens
message_stop	émet ChatEvent.Done (text, tokens, model, finishReason)

5. Tool framework

Chaque tool implémente l'interface ToolHandler et est **auto-découvert par Spring** à l'init via le ToolRegistry. **Règle d'or** : un tool ne touche JAMAIS la base de données directement — il délègue à un Service Spring existant qui porte les filtres tenant, l'autorisation et la logique métier.

Interface ToolHandler

```
public interface ToolHandler {
    String name(); // ex: "list_reservations"
    ToolDescriptor descriptor(); // JSON schema + requiresConfirmation
    ToolResult execute(JsonNode args, AgentContext context);
}
```

ToolResult (record)

ToolResult.success(jsonContent, displayHint) ou ToolResult.error(msg). Le displayHint route le rendu côté frontend vers le widget approprié.

displayHint	Widget React rendu
summary	KpiSummaryWidget (tuiles KPI horizontales)
list / table	DataTableWidget (table paginée)
chart-bar	BarChartWidget (Recharts vertical)
chart-line	LineChartWidget (évolution temporelle)
chart-pie	PieChartWidget (répartition)
insights	InsightsWidget (anomalies + recommandations)
navigation	NavigationCardWidget (bouton cliquable vers route)
portfolio	PortfolioOverviewWidget (KPIs cross-property)
weather	WeatherWidget (forecast 7j)
events	EventsWidget (jours fériés + festivals)
simulation	SimulationWidget (baseline vs scenario)
workflow	WorkflowWidget (étape multi-step)
knowledge	KnowledgeWidget (snippets RAG avec citations)

6. Inventaire des 27 tools

Répartition des 27 tools de l'Assistant IA

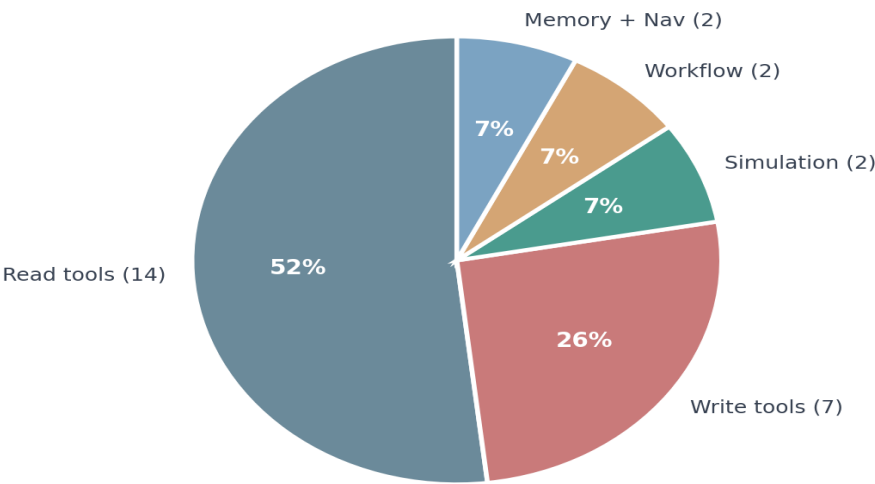


Figure 3 : Répartition des 27 tools par catégorie

6.1 Read tools (14 — sans confirmation)

Tool	Inputs	Cas d'usage
list_properties	city?, status?, type?, limit≤50	Liste propriétés
list_reservations	propertyId?, status?, dateRange?, limit	Réservations paginées
list_cleaning_tasks	propertyId?, status?, from?, limit	Tâches ménage
get_dashboard_summary	—	KPIs portfolio
get_financial_summary	propertyId?, months	Revenue/expense/profit bar chart
get_properties_performance	limit, metric (revenue/ratings)	Top N (bar chart)
get_business_insights	propertyId	Anomalies + recommandations
get_reservation_trend	propertyId, months	Line chart évolution
get_occupancy_forecast	propertyId, days≤90	% occupation prévisionnel
get_interventions_by_status	propertyId?, daysBack	Pie chart statuts
get_weather_forecast	city OR propertyId, days≤7	Météo (cache Redis 3h)

Tool	Inputs	Cas d'usage
get_local_events	city, from, to	Jours fériés + festivals + sport
analyze_portfolio	daysBack	KPIs cross-property + patterns
search_knowledge_base	query, topK≤10	Top K chunks doc Clenzy

6.2 Write tools (7 — avec confirmation utilisateur)

Tool	Inputs requis	Side-effect
create_intervention	propertyId, title, type, scheduledDate	INSERT + notif staff
assign_intervention	interventionId, assignedToUserId	UPDATE + notif assignee
cancel_reservation	reservationId, cancellationReason	Soft-delete + refund Stripe
update_property_status	propertyId, newStatus	Toggle online/offline OTA
send_guest_message	reservationId, templateId, variables	Envoi SMS/Email/WhatsApp
block_calendar_day	propertyId, from, to	INSERT BLOCKED + sync OTA
forget_fact	key	DELETE memory (irréversible)

■ Tous les **write tools** ont `requiresConfirmation=true`. Le LLM propose l'action, l'orchestrateur **pause le flux SSE**, le frontend affiche un dialog avec les arguments lisibles, et l'utilisateur confirme/refuse explicitement via `POST /api/assistant/tool-confirm`.

6.3 Simulation tools (2 — sans confirmation, read-only)

Tool	Inputs	Output
simulate_pricing_change	propertyId, pctChange ∈ [-50, +50], from?, to?	baseline vs scenario + recommandation textuelle
simulate_calendar_block	propertyId, from, to	revenue perdu + occupation break-even

6.4 Workflow + Memory + Navigation (4)

Tool	Rôle
start_workflow	Initie procédure guidée (onboard_property, end_of_month_closing...)
advance_workflow	Avance d'une étape après input user
remember_fact	Upsert memory (key, value, scope) + embedding
suggest_navigation	Bouton cliquable vers /route dans le chat

7. Knowledge Base RAG (pgvector)

Le **RAG (Retrieval-Augmented Generation)** permet à l'assistant de citer la documentation Clenzy avec un anti-hallucination strict. Les LLM ont une date de cutoff et ne connaissent pas la doc projet — le RAG comble cette lacune en injectant les passages pertinents directement dans le system prompt.

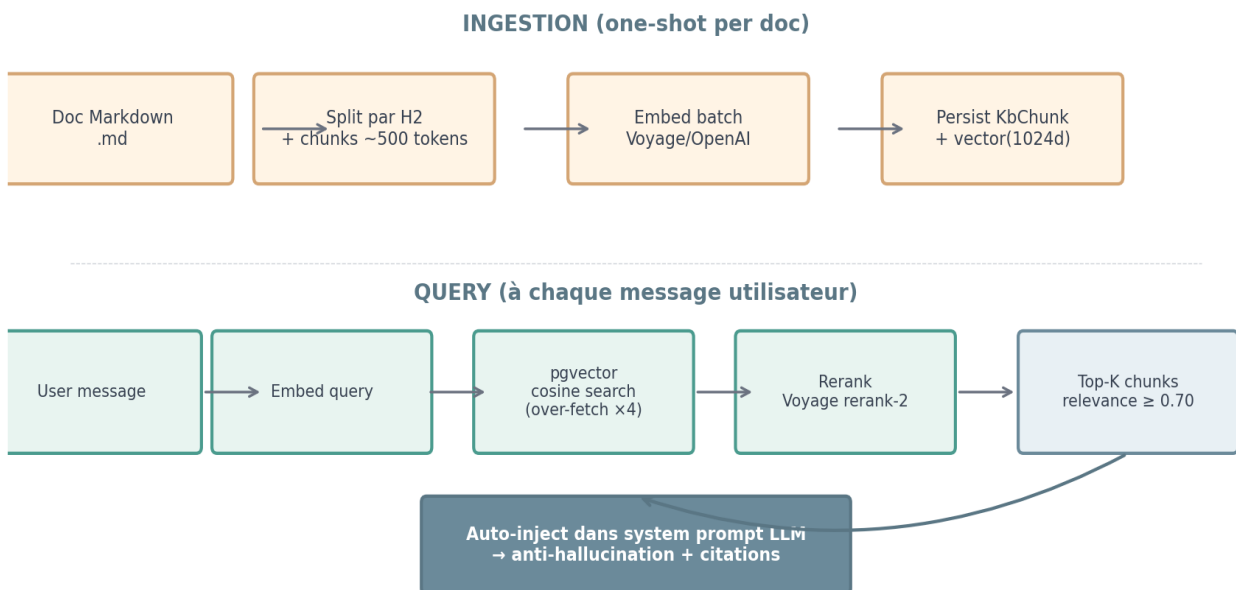


Figure 4 : Flow RAG complet — ingestion + query + auto-injection

Providers swappable

Provider	Modèle	Dim	Batch	Coût
Embeddings — Voyage	voyage-3-lite	1024	128 / batch	~\$0.02/1M tokens
Embeddings — OpenAI	text-embedding-3-small	1024 (param)	2048 / batch	~\$0.02/1M tokens
Re-rank — Voyage	rerank-2 (cross-encoder)	N/A	—	~\$0.05/1k pairs
Re-rank — NoOp	identity (fallback)	N/A	—	\$0

Auto-injection vs Tool explicit

Mode	Quand	Visibilité au LLM	Avantage
Auto-injection	À chaque message user	Invisible — chunks dans system prompt	Transparent

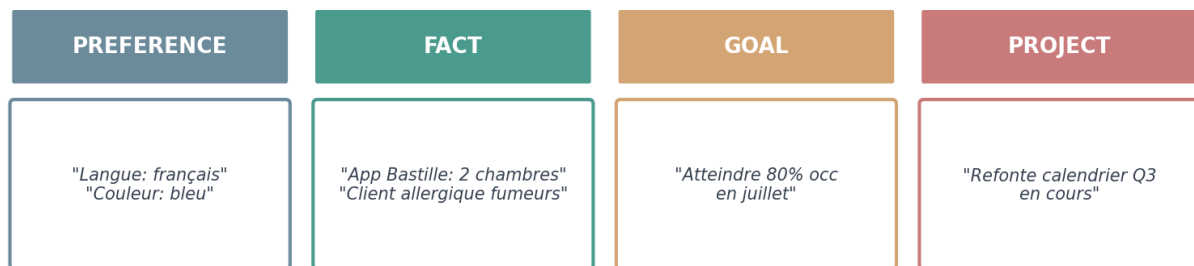
Mode	Quand	Visibilité au LLM	Avantage
Tool search_knowledge_base	LLM décide quand	Visible — résultat tool structuré	LLM contrôle topK

✓ **Anti-hallucination** : le system prompt contient l'instruction explicite "Cite tes sources via [titre](path) si tu utilises ces snippets — n'invente jamais une procédure non documentée". Combiné au modèle Haiku 4.5 strict pour les briefings, cela réduit drastiquement les hallucinations.

8. Memory long-terme

L'assistant retient durablement les préférences et faits importants de l'utilisateur, classés en **4 scopes**, sélectionnés par **pertinence sémantique** (embedding cosine similarity) à chaque message.

Memory long-terme : 4 scopes avec sélection par embedding



Cleanup automatique : `last_accessed_at > 6 mois` → `DELETE`

Figure 5 : 4 scopes de memory avec exemples concrets

Cascade de sélection

1. Si EmbeddingService disponible et message user non-blank → embed query → query native pgvector cosine sur les memories de l'user → top 30 triées par similarité.
2. Sinon (provider down ou message vide) → fallback listForUser triées par `last_accessed_at` desc.

Effet de bord : `last_accessed_at` est bumped en batch sur les memories retournées → purge naturelle des memories mortes (non lues depuis 6 mois → `DELETE`).

Cleanup automatique (lundi 3h UTC)

```
DELETE FROM assistant_memory
WHERE last_accessed_at < NOW() - INTERVAL '6 months'
OR (expires_at IS NOT NULL AND expires_at < NOW());
```

9. Simulations pricing & calendrier

SimulatePricingChangeTool

Question type : "Que se passe-t-il si je baisse le prix de 10% en juillet ?"

Algorithme :

```
ADR_baseline = revenue / nuits_occupées
ADR_scenario = ADR_baseline × (1 + pctChange)
Occupancy_baseline = nuits_occupées / jours_période
Occupancy_scenario = Occupancy_baseline × (1 - elasticity × pctChange)
Revenue_baseline = ADR_baseline × Occupancy_baseline × jours
Revenue_scenario = ADR_scenario × Occupancy_scenario × jours
Δ Revenue = Revenue_scenario - Revenue_baseline
Recommandation = "✓ Hausse recommandée" si Δ > 0 sinon "■ Baisse réduit le revenu"
```

EmpiricalElasticityEstimator

Calcule **empiriquement** l'élasticité prix-demande d'une propriété sur les 12 derniers mois en analysant les paires (mois T, mois T+1) avec $|\Delta \text{ADR}\%| \geq 2\%$. Au moins 3 paires significatives requises, élasticité bornée dans [0.1, 1.5]. Fallback : 0.5 (valeur industrie moyenne).

Optimisations performance

- **Bucketing $O(N \times M)$** par YearMonth plutôt que jour-par-jour → 100× plus rapide sur historiques chargés
- **Guards anti-div/0** : if (prevAdr ≤ 0) continue;, if (deltaAdr == 0) continue;
- **Cache** dans property_elasticity_estimate, recalcul scheduler weekly (> 30j obsolescence)

10. Workflows procéduraux

Les **workflows** sont des procédures multi-étapes guidées (ex: onboarding propriété, clôture de mois, préparation haute saison). Ils sont définis en YAML déclaratif et orchestrés par WorkflowEngine avec **optimistic locking** (@Version) pour la concurrence HA.

Définition YAML (exemple onboard_property)

```
id: onboard_property
name: "Onboarding nouvelle propriété"
steps:
  - id: type_property
    prompts:
      fr: "Quel type de bien ?"
      en: "What type of property?"
      ar: "■ ■ ■ ■ ■ ■ ■ ■ ■ ■"
    expects_data:
      type: string
      enum: [APARTMENT, HOUSE, STUDIO]
  - id: confirm
    prompts: { fr: "Confirmer la création de {{summary}} ?" }
    action: create_property
```

WorkflowEngine (sans état)

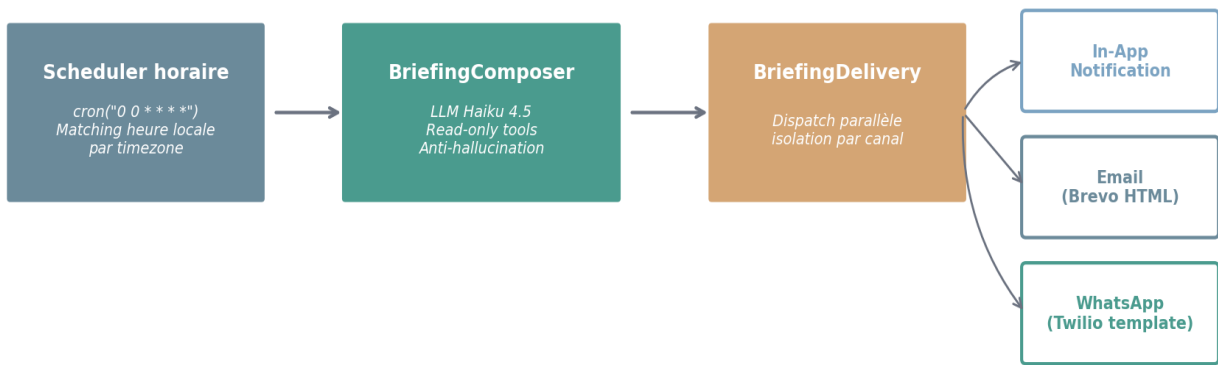
Méthode	Rôle
collectData(run, def, response)	Valide la réponse contre JSON schema, merge dans collected_data
advanceStep(run, def)	Incrémente current_step_idx ou marque COMPLETED
renderPrompt(step, run, language)	Interpole {{summary}}, {{collectedData.x}} dans la traduction
executeStepAction(step, run, language)	Retourne suggestion structurée pour le LLM (tool + args)

■ **Optimistic locking** : si deux requêtes tentent d'avancer le même run simultanément (user ouvre 2 onglets), seule l'une réussit. L'autre reçoit OptimisticLockException → l'agent peut re-prompter "réessayez". Évite les corruptions d'état.

11. Briefings proactifs

Recevoir chaque matin un **résumé personnalisé** des KPIs et alertes du portfolio, sans avoir à ouvrir le PMS. 3 canaux configurables par l'utilisateur : in-app, email HTML, WhatsApp (via Twilio template).

Briefings proactifs : flow scheduler → composition → dispatch 3 canaux



BriefingRetryScheduler horaire (CAS atomique) — retry des FAILED jusqu'à 24h

Figure 6 : Flow briefings — scheduler → composer (Haiku 4.5) → dispatch parallèle

Configuration utilisateur (Settings > IA > Briefings)

Champ	Options
Activé	on / off
Fréquence	DAILY_MORNING / WEEKLY_SUNDAY / ONLY_ALERTS
Heure locale	HH:mm (ex: 08:00)
Timezone	Europe/Paris (auto-déTECTÉ depuis JWT)
Canaux	multi-select [in_app, email, whatsapp]

Retry CAS atomique (BriefingRetryScheduler)

Le retry scheduler horaire utilise un **Compare-And-Swap atomique** sur le status du log : tryAcquireRetry(log_id, current=FAILED → new=RETRYING). Cela évite le double-retry si deux instances scheduler tournent en HA simultanément. Si l'acquisition échoue, l'instance skip ce log (une autre s'en occupe).

12. Vision (images)

Cas d'usage métier

- Upload photo de **dégradation** dans un logement → l'assistant crée automatiquement une intervention maintenance avec description
- Upload **facture fournisseur** → extraction des montants, fournisseur, date
- Upload **plan d'aménagement** → suggestions de mobilier

Limites Anthropic

Critère	Valeur
Taille max	5 MB par image
Formats acceptés	image/jpeg, image/png, image/gif, image/webp
Modèle requis	Claude 3.5 Sonnet+ (défaut claude-sonnet-4-20250514 ✓)
Limite UX	3 images max par message (frontend ChatInput.tsx)
Encoding	Base64 in-document (pas d'URLs)

Tracking & alertes

VisionTokenUsageService agrège les prompt_tokens des messages avec attachments IS NOT NULL sur 30 jours glissants par organisation. VisionUsageAlertScheduler compare l'usage vs le seuil configuré dans org_vision_alert.monthly_token_threshold et notifie l'admin org en cas de dépassement (avec cool-down via last_alerted_at pour éviter le spam).

■ **Sécurité ownership** : GET /api/assistant/attachments/{storageKey} valide via query native que le storageKey appartient bien à une conversation de l'utilisateur courant. **404 silencieux** si mismatch (évite l'énumération de storage keys).

13. Frontend chat

Arborescence

```
client/src/modules/assistant/  
  ■■■ AssistantPage.tsx (page racine, layout 3 zones)  
  ■■■ components/  
    ■ ■■■ MessageList.tsx (virtualized scroll)  
    ■ ■■■ MessageBubble.tsx (user droite / assistant gauche)  
    ■ ■■■ ChatInput.tsx (textarea + voice + upload)  
    ■ ■■■ ToolCallCard.tsx  
    ■ ■■■ ToolConfirmationDialog.tsx  
    ■ ■■■ ConversationListSidebar.tsx  
  ■■■ widgets/  
    ■■■ ToolResultWidget.tsx (routeur par displayHint)  
    ■■■ 13 widgets spécialisés (KpiSummary, DataTable, Charts...)
```

Voice input (Web Speech API)

Aspect	Valeur
API native	SpeechRecognition (Chrome/Edge/Safari)
Mapping langue	fr-FR, en-US, ar-MA (depuis i18n context)
continuous	false (1 utterance par activation)
interimResults	true (feedback temps réel)
Comportement	Append au texte existant (pas d'overwrite)
Animation	Pulse mic CSS keyframes pendant écoute
Errors gérées	not-allowed, audio-capture, network, no-speech

14. Sécurité & multi-tenant

Couche	Mécanisme
Authentification	Keycloak JWT (cookie HttpOnly clenzy_auth)
Autorisation controller	@PreAuthorize("isAuthenticated()") sur AssistantController
Autorisation tools	Délégation aux Services Spring (checks rôles + ownership)
Multi-tenant	TenantFilter → TenantContext (ThreadLocal) → Hibernate organizationFilter
Confirmation write tools	requiresConfirmation=true → POST /tool-confirm explicite
Audit	AuditLogService trace toutes les actions write tools
API keys	Plateforme : env var. BYOK : AES-256 in org_ai_api_key. Jamais loggées en clair.
Vision attachments	Native query ownership check (404 si mismatch, anti-énumération)

■ **Cloisonnement strict** : aucune fuite cross-organisation possible via les repositories JPA standards (filtre Hibernate automatique). Les *native queries* sont auditées manuellement pour inclure WHERE organization_id = :orgId explicite.

15. Configuration & déploiement

Variables d'environnement (plateforme)

```
# LLM
ANTHROPIC_API_KEY=sk-ant-... # fallback BYOK

# Embeddings
clenzy.ai.embeddings.provider=voyage # ou openai
clenzy.ai.embeddings.voyage.api-key=pa-...
clenzy.ai.embeddings.relevance-threshold=0.70

# Re-ranking
clenzy.ai.rerank.enabled=true
clenzy.ai.rerank.provider=voyage # ou noop

# Memory
clenzy.assistant.memory.relevance-enabled=true
clenzy.assistant.memory.cleanup-enabled=true

# Briefings
clenzy.assistant.briefing.retry-enabled=true
clenzy.assistant.briefing.model=claude-haiku-4-5

# Vision
clenzy.assistant.vision.enabled=true
```

Schedulers actifs (6)

Scheduler	Cron	Rôle
BriefingScheduler	0 0 * * * (horaire)	Émet briefings selon prefs user
BriefingRetryScheduler	0 30 * * * (horaire+30)	Retry des briefings FAILED (CAS atomique)
AssistantMemoryCleanupScheduler	0 0 3 * * 1 (lundi 3h UTC)	Purge memories stales > 6 mois
VisionUsageAlertScheduler	0 0 9 * * 1 (lundi 9h)	Alerte si org dépasse son quota vision
ElasticityRecomputeScheduler	0 0 4 * * 0 (dim 4h)	Recalcule élasticité empirique > 30j
KbIndexTuningScheduler	0 0 2 1 * * (1er du mois)	Recalcule ivfflat lists selon volume

■ **Docker pgvector requis** : l'image PostgreSQL doit être pgvector/pgvector:pg15 (pas postgres:15 standard) — sinon la migration 0143 (CREATE EXTENSION vector) échoue au boot. Voir clenzy-infra/docker-compose.dev.yml.

16. Limites connues & TODOs

Implémenté mais perfectible

- **Token budget non-enforcé**

AiProperties.TokenBudget.enforced=true configurable mais pas de logique de blocage dans AgentOrchestrator (passive monitoring uniquement).

- **PendingToolStore in-memory**

Les tools en attente de confirmation expirent à la fin de la session (pas de persistance BDD). Si user ferme l'onglet, le tool est perdu.

- **Élasticité sans saisonnalité**

Moyenne arithmétique des paires $(T, T+1)$, peut être biaisée par un mois exceptionnel. À terme : régression linéaire multi-variée.

- **Pas de retry exponentiel Anthropic**

Erreur réseau → propagée au frontend qui peut re-envoyer. Le SDK HTTP a un retry basique mais non configuré.

- **Workflows non hot-reloadables**

Les définitions YAML sont chargées au boot via @PostConstruct. Modification nécessite restart.

Roadmap potentielle

- Streaming voice output (TTS Eleven Labs pour réponses parlées, mobile-first)
- Multi-tour confirmation ("tu confirmes ces 5 interventions ?" en batch)
- Memory hiérarchique (org-shared memory en plus de user-scoped)
- Tool sandboxing (rate limit par user, max writes/jour)
- A/B testing prompts (framework pour comparer 2 system prompts en prod)
- Fine-tuning du re-ranker sur les data Clenzy (cross-encoder spécifique)

Coûts opérationnels (estimation 1000 utilisateurs actifs)

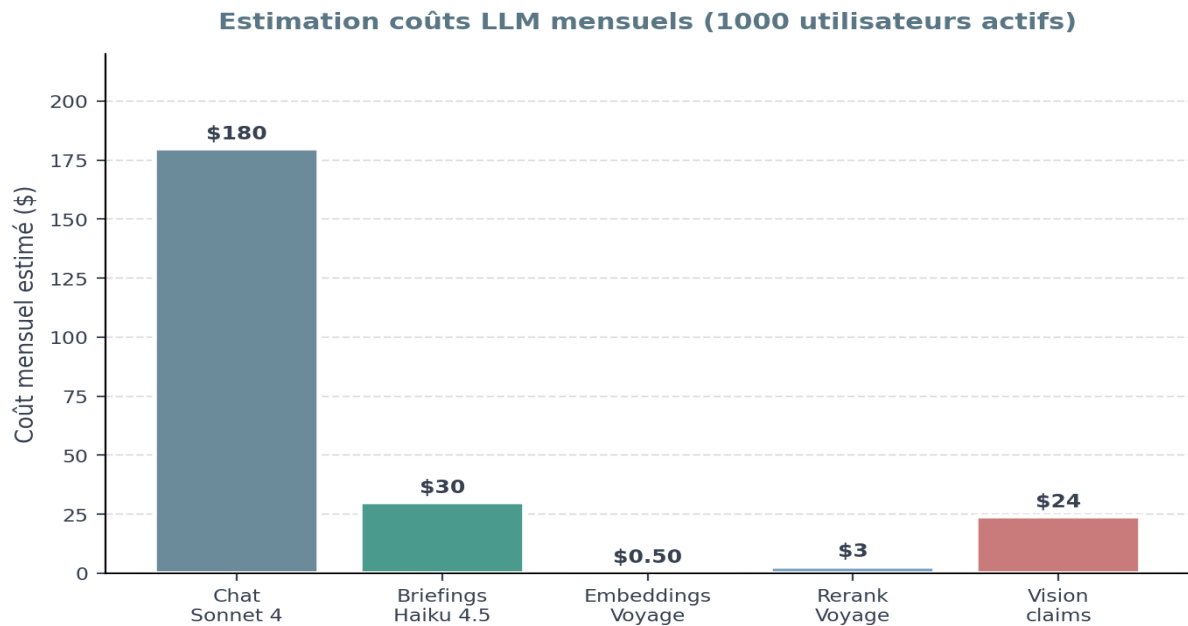


Figure 7 : Estimation des coûts LLM mensuels par usage type

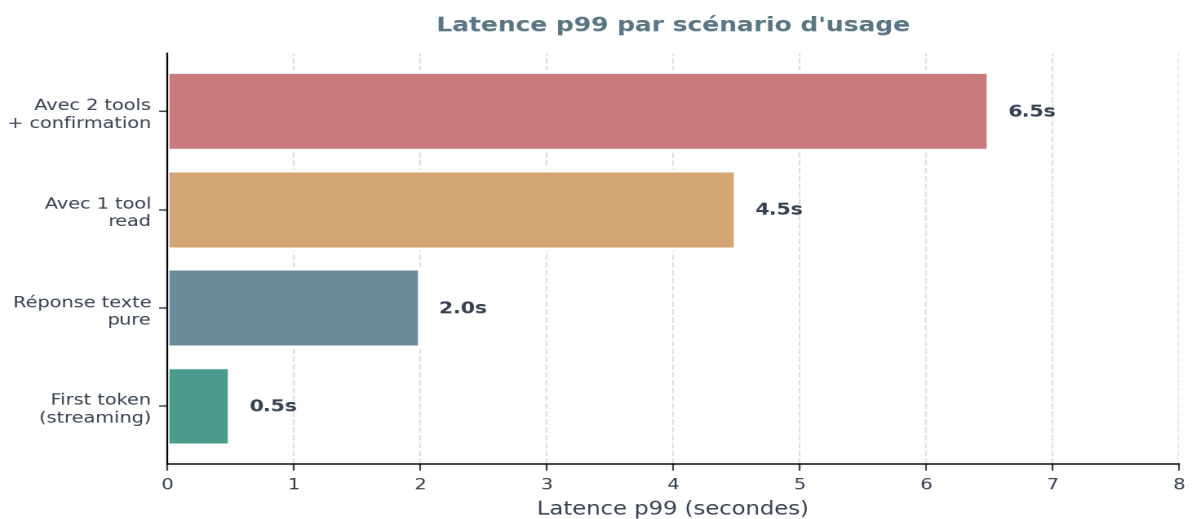


Figure 8 : Latence p99 par scénario d'usage

■ **Quota Anthropic** : tier 4 par défaut (4000 RPM, 400k TPM). Suffisant pour ~100 users actifs simultanés. Au-delà : passer en tier 5 ou implémenter du queueing.

Fin du document — pour questions ou évolutions, contacter l'équipe Clenzy.